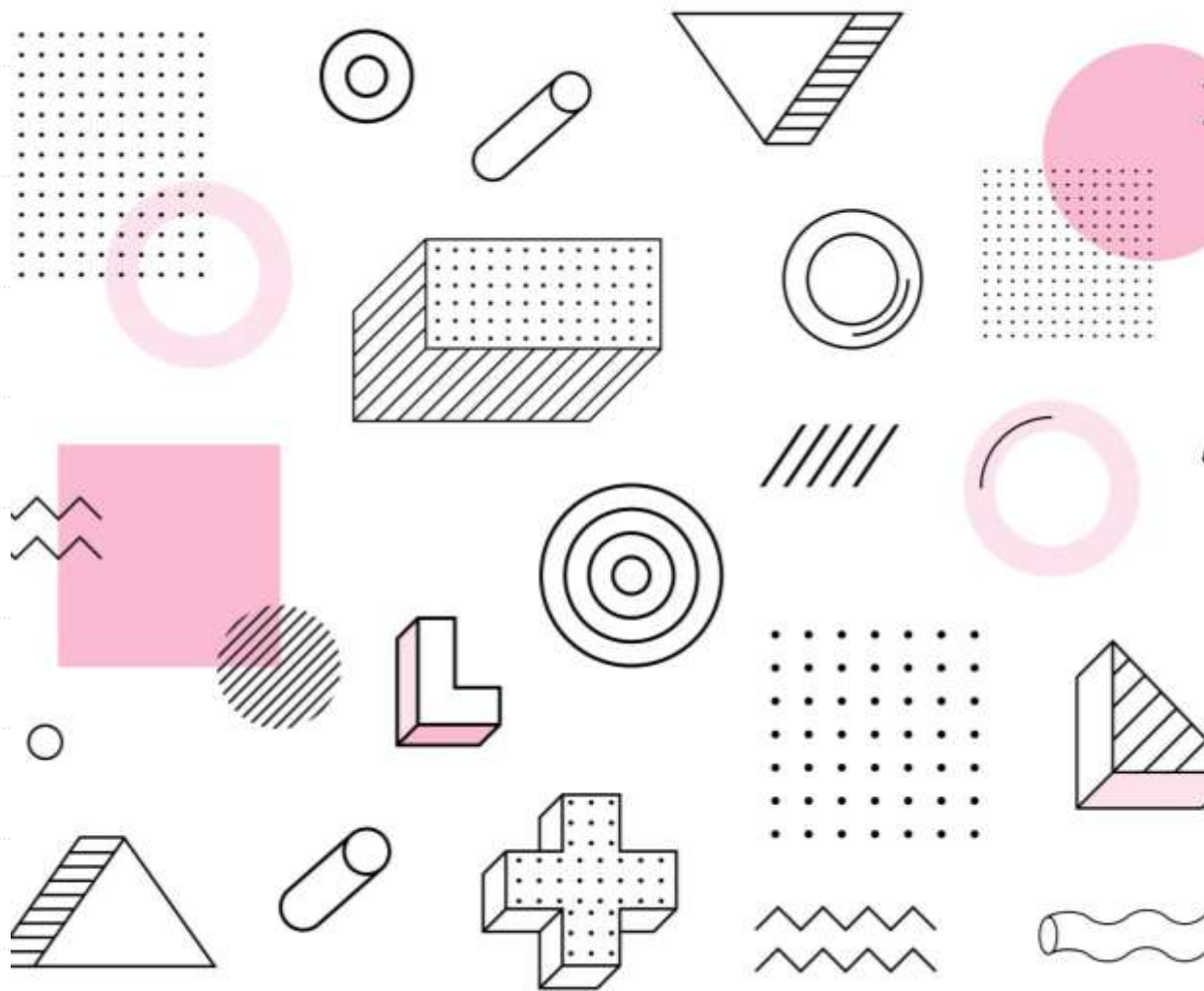


Chapter 6: Bottom-Up Parsing (Shift-Reduce)

陳奇業 成功大學資訊工程系



LALR(k) Table Construction

1 Start $\rightarrow$ S \$
 2 S $\rightarrow$ A B
 3 | a c
 4 | x A c
 5 A $\rightarrow$ a
 6 B $\rightarrow$ b
 7 | λ

State 0	Goto
Start $\rightarrow \bullet$ S \$	4
S $\rightarrow \bullet$ A B	2
S $\rightarrow \bullet$ a c	3
S $\rightarrow \bullet$ x A c	1
A $\rightarrow \bullet$ a	3

State 3	Goto
S $\rightarrow$ a $\bullet$ c	6
A $\rightarrow$ a $\bullet$	

$Follow(A) = \{c, b, \$\}$

shift/reduce conflict

LALR(k) Table Construction

1 Start $\rightarrow$ S \$
 2 S $\rightarrow$ A₁ B
 3 | a c
 4 | x A₂ c
 5 A₁ $\rightarrow$ a
 6 A₂ $\rightarrow$ a
 7 B $\rightarrow$ b
 8 | λ

State 0	Goto
Start $\rightarrow \bullet$ S \$	3
S $\rightarrow \bullet$ A ₁ B	4
S $\rightarrow \bullet$ a c	2
S $\rightarrow \bullet$ x A ₂ c	1
A ₁ $\rightarrow \bullet$ a	2

State 2	Goto
S $\rightarrow$ a $\bullet$ c	8
A ₁ $\rightarrow$ a $\bullet$	

Case 1:

$Start \rightarrow S\$ \rightarrow A_1B\$ \rightarrow A_1b\$$ *Follow(A₁) = {b, \$}*

$Start \rightarrow S\$ \rightarrow A_1B\$ \rightarrow A_1\$$

Case 2:

$Start \rightarrow S\$ \rightarrow xA_2c\$$ *Follow(A₂) = {c}*



LALR(k) Table Construction

- In this section, we consider LALR(k) (Lookahead Ahead LR with k tokens of lookahead) parsing, which offers a more specialized computation of the symbols that can follow a nonterminal.
- LALR offers superior lookahead analysis for constructing the bottom-up parsing table.
- LALR(1) parsers can be built by first constructing an LR(1) parser and then merging states



LALR(k) Table Construction

- LALR(1) parsers can be built by
 1. An LR(1) parser and then merging states (may be quite inefficient)
 2. An LR(0) parser with LALR propagation graph

LALR(k) Table Construction

```
procedure COMPLETETABLE(Table, grammar)  
  call COMPUTELOOKAHEAD( )  
  foreach state  $\in$  Table do  
    foreach rule  $\in$  Productions(grammar) do  
      call TRYRULEINSTATE(state, rule)  
    call ASSERTENTRY(StartState, GoalSymbol, accept)  
  end  
procedure ASSERTENTRY(state, symbol, action)  
  if Table[state][symbol] = error  
  then Table[state][symbol]  $\leftarrow$  action  
  else  
    call REPORTCONFLICT(Table[state][symbol], action)  
  end  
end
```

```
procedure TRYRULEINSTATE(s, r)  
  if  $\text{LHS}(r) \rightarrow \text{RHS}(r) \bullet \in s$   
  then  
    foreach  $\mathcal{X} \in \text{Follow}(\text{LHS}(r))$  do  
      call ASSERTENTRY(s,  $\mathcal{X}$ , reduce r)  
    end  
  end
```



```
procedure TRYRULEINSTATE(s, r)  
  if  $\text{LHS}(r) \rightarrow \text{RHS}(r) \bullet \in s$   
  then  
    foreach  $\mathcal{X} \in \Sigma$  do  
      if  $\mathcal{X} \in \text{ItemFollow}((s, \text{LHS}(r) \rightarrow \text{RHS}(r) \bullet))$   
      then call ASSERTENTRY(s,  $\mathcal{X}$ , reduce r)  
    end  
  end
```



LALR(k) Table Construction

```
procedure COMPUTELOOKAHEAD( )  
    call BUILDITEMPROPGRAPH( )  
    call EVALITEMPROPGRAPH( )  
end
```


LALR Propagation Graph

- We have not formally named each LR(0) item, but an item occurs at most once in any state. Thus, the pair $(s, A \rightarrow \alpha \bullet \beta)$ suffices to identify an item $A \rightarrow \alpha \bullet \beta$ that occurs in state s .
- For each valid state and item pair, we create a vertex v in the LALR propagation graph.

```
procedure BUILDITEMPROPGRAPH()  
  foreach  $s \in \text{States}$  do  
    foreach  $\text{item} \in \text{state}$  do  
       $v \leftarrow \text{Graph.ADDVERTEX}(s, \text{item})$   
       $\text{ItemFollow}(v) \leftarrow \emptyset$   
    foreach  $p \in \text{PRODUCTIONSFOR}(\text{Start})$  do  
       $\text{ItemFollow}((\text{StartState}, \text{Start} \rightarrow \bullet \text{RHS}(p))) \leftarrow \{\$ \}$   
    foreach  $s \in \text{States}$  do  
      foreach  $A \rightarrow \alpha \bullet B\gamma \in s$  do  
         $v \leftarrow \text{Graph.FINDVERTEX}(s, A \rightarrow \alpha \bullet B\gamma)$   
        call  $\text{Graph.ADDEDGE}(v, (\text{Table}[s][B], A \rightarrow \alpha B \bullet \gamma))$   
        foreach  $(w \leftarrow (s, B \rightarrow \bullet \delta)) \in \text{Graph.Vertices}$  do  
           $\text{ItemFollow}(w) \leftarrow \text{ItemFollow}(w) \cup \text{First}(\gamma)$   
          if  $\text{ALLDERIVEEMPTY}(\gamma)$   
          then call  $\text{Graph.ADDEDGE}(v, w)$   
      end  
    end  
  end  
end
```


LALR Propagation Graph

- The ItemFollow sets are initially empty, except for the augmenting item $\text{Start} \rightarrow \bullet S \$$ in the LR(0) start-state.

```
procedure BUILDITEMPROPGRAPH()  
  foreach  $s \in \text{States}$  do  
    foreach  $\text{item} \in \text{state}$  do  
       $v \leftarrow \text{Graph.ADDVERTEX}(s, \text{item})$   
       $\text{ItemFollow}(v) \leftarrow \emptyset$   
    foreach  $p \in \text{PRODUCTIONSFOR}(\text{Start})$  do  
       $\text{ItemFollow}((\text{StartState}, \text{Start} \rightarrow \bullet \text{RHS}(p))) \leftarrow \{\$ \}$   
    foreach  $s \in \text{States}$  do  
      foreach  $A \rightarrow \alpha \bullet B \gamma \in s$  do  
         $v \leftarrow \text{Graph.FINDVERTEX}(s, A \rightarrow \alpha \bullet B \gamma)$   
        call  $\text{Graph.ADDEDGE}(v, (\text{Table}[s][B], A \rightarrow \alpha B \bullet \gamma))$   
        foreach  $(w \leftarrow (s, B \rightarrow \bullet \delta)) \in \text{Graph.Vertices}$  do  
           $\text{ItemFollow}(w) \leftarrow \text{ItemFollow}(w) \cup \text{First}(\gamma)$   
          if ALLDERIVEEMPTY( $\gamma$ )  
          then call  $\text{Graph.ADDEDGE}(v, w)$   
      end  
    end  
  end  
end
```

LALR Propagation Graph

- Edges are placed in the graph between items i and j when the symbols that follow the reducible form of item i should be included in the corresponding set of symbols for item j .

```
procedure BUILDITEMPROPGRAPH()  
  foreach  $s \in States$  do  
    foreach  $item \in state$  do  
       $v \leftarrow Graph.ADDVERTEX((s, item))$   
       $ItemFollow(v) \leftarrow \emptyset$   
    foreach  $p \in PRODUCTIONSFOR(Start)$  do  
       $ItemFollow((StartState, Start \rightarrow \bullet RHS(p))) \leftarrow \{\$ \}$   
    foreach  $s \in States$  do  
      foreach  $A \rightarrow \alpha \bullet B \gamma \in s$  do  
         $v \leftarrow Graph.FINDVERTEX((s, A \rightarrow \alpha \bullet B \gamma))$   
        call  $Graph.ADDEDGE(v, (Table[s][B], A \rightarrow \alpha B \bullet \gamma))$   
        foreach  $(w \leftarrow (s, B \rightarrow \bullet \delta)) \in Graph.Vertices$  do  
           $ItemFollow(w) \leftarrow ItemFollow(w) \cup First(\gamma)$   
          if ALLDERIVEEMPTY( $\gamma$ )  
          then call  $Graph.ADDEDGE(v, w)$   
      end  
    end  
  end  
end
```

LALR Propagation Graph

- For the item $A \rightarrow \alpha \bullet B \gamma$, any symbol in $\text{First}(\gamma)$ can follow each closure item $B \rightarrow \bullet \delta$.

```
procedure BUILDITEMPROPGRAPH()  
  foreach  $s \in \text{States}$  do  
    foreach  $\text{item} \in \text{state}$  do  
       $v \leftarrow \text{Graph.ADDVERTEX}(s, \text{item})$   
       $\text{ItemFollow}(v) \leftarrow \emptyset$   
    foreach  $p \in \text{PRODUCTIONSFOR}(\text{Start})$  do  
       $\text{ItemFollow}((\text{StartState}, \text{Start} \rightarrow \bullet \text{RHS}(p))) \leftarrow \{\$ \}$   
    foreach  $s \in \text{States}$  do  
      foreach  $A \rightarrow \alpha \bullet B \gamma \in s$  do  
         $v \leftarrow \text{Graph.FINDVERTEX}(s, A \rightarrow \alpha \bullet B \gamma)$   
        call  $\text{Graph.ADDEDGE}(v, (\text{Table}[s][B], A \rightarrow \alpha B \bullet \gamma))$   
        foreach  $(w \leftarrow (s, B \rightarrow \bullet \delta)) \in \text{Graph.Vertices}$  do  
           $\text{ItemFollow}(w) \leftarrow \text{ItemFollow}(w) \cup \text{First}(\gamma)$   
          if ALLDERIVEEMPTY( $\gamma$ )  
          then call  $\text{Graph.ADDEDGE}(v, w)$   
        end  
      end  
    end  
  end  
end
```

LALR Propagation Graph

- Consider again the item $A \rightarrow \alpha \bullet B \gamma$ and the closure items introduced when B is a nonterminal. When $\gamma \Rightarrow^* \lambda$, either because γ is absent or because the string of symbols in γ can derive λ , then any symbol that can follow A can also follow B .

```
procedure BUILDITEMPROPGRAPH()  
  foreach  $s \in \text{States}$  do  
    foreach  $\text{item} \in \text{state}$  do  
       $v \leftarrow \text{Graph.ADDVERTEX}(s, \text{item})$   
       $\text{ItemFollow}(v) \leftarrow \emptyset$   
    foreach  $p \in \text{PRODUCTIONSFOR}(\text{Start})$  do  
       $\text{ItemFollow}((\text{StartState}, \text{Start} \rightarrow \bullet \text{RHS}(p))) \leftarrow \{\$ \}$   
    foreach  $s \in \text{States}$  do  
      foreach  $A \rightarrow \alpha \bullet B \gamma \in s$  do  
         $v \leftarrow \text{Graph.FINDVERTEX}(s, A \rightarrow \alpha \bullet B \gamma)$   
        call  $\text{Graph.ADDEDGE}(v, (\text{Table}[s][B], A \rightarrow \alpha B \bullet \gamma))$   
        foreach  $(w \leftarrow (s, B \rightarrow \bullet \delta)) \in \text{Graph.Vertices}$  do  
           $\text{ItemFollow}(w) \leftarrow \text{ItemFollow}(w) \cup \text{First}(\gamma)$   
          if ALLDERIVEEMPTY( $\gamma$ )  
            then call  $\text{Graph.ADDEDGE}(v, w)$   
        end  
      end  
    end  
  end  
end
```

LALR Propagation Graph

```

procedure BUILDITEMPROPGRAPH( )
  foreach  $s \in \text{States}$  do
    foreach  $\text{item} \in \text{state}$  do
       $v \leftarrow \text{Graph.ADDVERTEX}(s, \text{item})$ 
       $\text{ItemFollow}(v) \leftarrow \emptyset$ 
    foreach  $p \in \text{PRODUCTIONSFOR}(\text{Start})$  do
       $\text{ItemFollow}((\text{StartState}, \text{Start} \rightarrow \bullet \text{RHS}(p))) \leftarrow \{\$ \}$ 
    foreach  $s \in \text{States}$  do
      foreach  $A \rightarrow \alpha \bullet B \gamma \in s$  do
         $v \leftarrow \text{Graph.FINDVERTEX}(s, A \rightarrow \alpha \bullet B \gamma)$ 
        call  $\text{Graph.ADDEDGE}(v, (\text{Table}[s][B], A \rightarrow \alpha B \bullet \gamma))$ 
        foreach  $(w \leftarrow (s, B \rightarrow \bullet \delta)) \in \text{Graph.Vertices}$  do
           $\text{ItemFollow}(w) \leftarrow \text{ItemFollow}(w) \cup \text{First}(\gamma)$ 
          if  $\text{ALLDERIVEEMPTY}(\gamma)$ 
            then call  $\text{Graph.ADDEDGE}(v, w)$ 
  end

```

State	LR(0) Item	Goto State	Prop Edges Placed by Step		Initialize ItemFollow First(γ)	
			(27)	(29)	(28)	(28)
0	1 Start $\rightarrow \bullet$ S \$	4	13		\$	2,3,4
	2 S $\rightarrow \bullet$ A B	2	8	5	b	5
	3 S $\rightarrow \bullet$ a c	3	11			
	4 S $\rightarrow \bullet$ x A c	1	6			
	5 A $\rightarrow \bullet$ a	3	12			
1	6 S $\rightarrow$ x $\bullet$ A c	9	18		c	7
	7 A $\rightarrow \bullet$ a	10	19			
2	8 S $\rightarrow$ A $\bullet$ B	8	17	9,10		
	9 B $\rightarrow \bullet$ b	7	16			
	10 B $\rightarrow \bullet$					
3	11 S $\rightarrow$ a $\bullet$ c	6	15			
	12 A $\rightarrow$ a $\bullet$					
4	13 Start $\rightarrow$ S $\bullet$ \$	5	14			
5	14 Start $\rightarrow$ S $\bullet$					
6	15 S $\rightarrow$ a c $\bullet$					
7	16 B $\rightarrow$ b $\bullet$					
8	17 S $\rightarrow$ A B $\bullet$					
9	18 S $\rightarrow$ x A $\bullet$ c	11	20			
10	19 A $\rightarrow$ a $\bullet$					
11	20 S $\rightarrow$ x A c $\bullet$					

LALR Propagation Graph

```

procedure BUILDITEMPROPGRAPH( )
  foreach  $s \in \text{States}$  do
    foreach  $\text{item} \in \text{state}$  do
       $v \leftarrow \text{Graph.ADDVERTEX}((s, \text{item}))$ 
       $\text{ItemFollow}(v) \leftarrow \emptyset$ 
    foreach  $p \in \text{PRODUCTIONSFOR}(\text{Start})$  do
       $\text{ItemFollow}((\text{StartState}, \text{Start} \rightarrow \bullet \text{RHS}(p))) \leftarrow \{\$ \}$ 
    foreach  $s \in \text{States}$  do
      foreach  $A \rightarrow \alpha \bullet B \gamma \in s$  do
         $v \leftarrow \text{Graph.FINDVERTEX}((s, A \rightarrow \alpha \bullet B \gamma))$ 
        call  $\text{Graph.ADDEDGE}(v, (\text{Table}[s][B], A \rightarrow \alpha B \bullet \gamma))$ 
        foreach  $(w \leftarrow (s, B \rightarrow \bullet \delta)) \in \text{Graph.Vertices}$  do
           $\text{ItemFollow}(w) \leftarrow \text{ItemFollow}(w) \cup \text{First}(\gamma)$ 
          if  $\text{ALLDERIVEEMPTY}(\gamma)$ 
          then call  $\text{Graph.ADDEDGE}(v, w)$ 
      end
    end
  end

```

State	LR(0) Item	Goto State	Prop Edges Placed by Step		Initialize ItemFollow First(γ)	
			(27)	(29)	(28)	(28)
0	1 $\text{Start} \rightarrow \bullet S \$$	4	13		\$	2,3,4
	2 $S \rightarrow \bullet A B$	2	8	5	b	5
	3 $S \rightarrow \bullet a c$	3	11			
	4 $S \rightarrow \bullet x A c$	1	6			
	5 $A \rightarrow \bullet a$	3	12			
1	6 $S \rightarrow x \bullet A c$	9	18		c	7
	7 $A \rightarrow \bullet a$	10	19			
2	8 $S \rightarrow A \bullet B$	8	17	9,10		
	9 $B \rightarrow \bullet b$	7	16			
	10 $B \rightarrow \bullet$					
3	11 $S \rightarrow a \bullet c$	6	15			
	12 $A \rightarrow a \bullet$					
4	13 $\text{Start} \rightarrow S \bullet \$$	5	14			
5	14 $\text{Start} \rightarrow S \$ \bullet$					
6	15 $S \rightarrow a c \bullet$					
7	16 $B \rightarrow b \bullet$					
8	17 $S \rightarrow A B \bullet$					
9	18 $S \rightarrow x A \bullet c$	11	20			
10	19 $A \rightarrow a \bullet$					
11	20 $S \rightarrow x A c \bullet$					

LALR Propagation Graph

```

1 Start → S $
2 S    → A B
3      | a c
4      | x A c
5 A    → a
6 B    → b
7      | λ
    
```

```

procedure BUILDITEMPROPGRAPH( )
  foreach  $s \in \text{States}$  do
    foreach  $\text{item} \in \text{state}$  do
       $v \leftarrow \text{Graph.ADDVERTEX}(s, \text{item})$ 
       $\text{ItemFollow}(v) \leftarrow \emptyset$ 
    foreach  $p \in \text{PRODUCTIONSFOR}(\text{Start})$  do
       $\text{ItemFollow}((\text{StartState}, \text{Start} \rightarrow \bullet \text{RHS}(p))) \leftarrow \{\$ \}$ 
    foreach  $s \in \text{States}$  do
      foreach  $A \rightarrow \alpha \bullet B \gamma \in s$  do
         $v \leftarrow \text{Graph.FINDVERTEX}(s, A \rightarrow \alpha \bullet B \gamma)$ 
        call  $\text{Graph.ADDEDGE}(v, (\text{Table}[s][B], A \rightarrow \alpha B \bullet \gamma))$ 
        foreach  $(w \leftarrow (s, B \rightarrow \bullet \delta)) \in \text{Graph.Vertices}$  do
           $\text{ItemFollow}(w) \leftarrow \text{ItemFollow}(w) \cup \text{First}(\gamma)$ 
          if ALLDERIVEEMPTY( $\gamma$ )
            then call  $\text{Graph.ADDEDGE}(v, w)$ 
        end
      end
    end
  end
    
```

State	LR(0) Item	Goto State	Prop Edges Placed by Step		Initialize ItemFollow First(γ)	
			(27)	(29)	(28)	(28)
0	1 Start $\rightarrow \bullet$ S \$	4	13	5	\$	2,3,4
	2 S $\rightarrow \bullet$ A B	2	8		b	5
	3 S $\rightarrow \bullet$ a c	3	11			
	4 S $\rightarrow \bullet$ x A c	1	6			
	5 A $\rightarrow \bullet$ a	3	12			
1	6 S $\rightarrow$ x $\bullet$ A c	9	18	9,10	c	7
	7 A $\rightarrow \bullet$ a	10	19			
2	8 S $\rightarrow$ A $\bullet$ B	8	17			
	9 B $\rightarrow \bullet$ b	7	16			
	10 B $\rightarrow \bullet$					
3	11 S $\rightarrow$ a $\bullet$ c	6	15			
	12 A $\rightarrow$ a $\bullet$					
4	13 Start $\rightarrow$ S $\bullet$ \$	5	14			
5	14 Start $\rightarrow$ S $\bullet$					
6	15 S $\rightarrow$ a c $\bullet$					
7	16 B $\rightarrow$ b $\bullet$					
8	17 S $\rightarrow$ A B $\bullet$					
9	18 S $\rightarrow$ x A $\bullet$ c	11	20			
10	19 A $\rightarrow$ a $\bullet$					
11	20 S $\rightarrow$ x A c $\bullet$					

LALR Propagation Graph

```

1 Start → S $
2 S      → A B
3        | a c
4        | x A c
5 A      → a
6 B      → b
7        | λ
    
```

```

procedure BUILDITEMPROPGRAPH( )
  foreach  $s \in \text{States}$  do
    foreach  $\text{item} \in \text{state}$  do
       $v \leftarrow \text{Graph.ADDVERTEX}((s, \text{item}))$ 
       $\text{ItemFollow}(v) \leftarrow \emptyset$ 
    foreach  $p \in \text{PRODUCTIONSFOR}(\text{Start})$  do
       $\text{ItemFollow}((\text{StartState}, \text{Start} \rightarrow \bullet \text{RHS}(p))) \leftarrow \{\$ \}$ 
    foreach  $s \in \text{States}$  do
      foreach  $A \rightarrow \alpha \bullet B \gamma \in s$  do
         $v \leftarrow \text{Graph.FINDVERTEX}((s, A \rightarrow \alpha \bullet B \gamma))$ 
        call  $\text{Graph.ADDEDGE}(v, (\text{Table}[s][B], A \rightarrow \alpha B \bullet \gamma))$ 
        foreach  $(w \leftarrow (s, B \rightarrow \bullet \delta)) \in \text{Graph.Vertices}$  do
           $\text{ItemFollow}(w) \leftarrow \text{ItemFollow}(w) \cup \text{First}(\gamma)$ 
          if  $\text{ALLDERIVEEMPTY}(\gamma)$ 
            then call  $\text{Graph.ADDEDGE}(v, w)$ 
        end
      end
    end
  end
    
```

State	LR(0) Item	Goto State	Prop Edges Placed by Step		Initialize ItemFollow First(γ)	
			(27)	(29)	(28)	(28)
0	1 Start $\rightarrow \bullet$ S \$	4	13		\$	2,3,4
	2 S $\rightarrow \bullet$ A B	2	8	5	b	5
	3 S $\rightarrow \bullet$ a c	3	11			
	4 S $\rightarrow \bullet$ x A c	1	6			
	5 A $\rightarrow \bullet$ a	3	12			
1	6 S $\rightarrow$ x $\bullet$ A c	9	18		c	7
	7 A $\rightarrow \bullet$ a	10	19			
2	8 S $\rightarrow$ A $\bullet$ B	8	17	9,10		
	9 B $\rightarrow \bullet$ b	7	16			
	10 B $\rightarrow \bullet$					
3	11 S $\rightarrow$ a $\bullet$ c	6	15			
	12 A $\rightarrow$ a $\bullet$					
4	13 Start $\rightarrow$ S $\bullet$ \$	5	14			
5	14 Start $\rightarrow$ S \$ $\bullet$					
6	15 S $\rightarrow$ a c $\bullet$					
7	16 B $\rightarrow$ b $\bullet$					
8	17 S $\rightarrow$ A B $\bullet$					
9	18 S $\rightarrow$ x A $\bullet$ c	11	20			
10	19 A $\rightarrow$ a $\bullet$					
11	20 S $\rightarrow$ x A c $\bullet$					

LALR Propagation Graph

```
procedure EVALITEMPROPGRAPH( )  
  repeat  
    changed  $\leftarrow$  false  
    foreach  $(v, w) \in \text{Graph.Edges}$  do  
      old  $\leftarrow$  ItemFollow(w)  
      ItemFollow(w)  $\leftarrow$  ItemFollow(w)  $\cup$  ItemFollow(v)  
      if ItemFollow(w)  $\neq$  old  
      then changed  $\leftarrow$  true  
    until not changed  
  end
```

LALR Propagation Graph

State	LR(0) Item	Goto State	Prop Edges Placed by Step		Initialize ItemFollow First(γ)	
			(27)	(29)	(28)	
0	1 Start $\rightarrow \bullet$ S \$	4	13		\$	2,3,4
	2 S $\rightarrow \bullet$ A B	2	8	5	b	5
	3 S $\rightarrow \bullet$ a c	3	11			
	4 S $\rightarrow \bullet$ x A c	1	6			
	5 A $\rightarrow \bullet$ a	3	12			
1	6 S $\rightarrow$ x $\bullet$ A c	9	18		c	7
	7 A $\rightarrow \bullet$ a	10	19			
2	8 S $\rightarrow$ A $\bullet$ B	8	17	9,10		
	9 B $\rightarrow \bullet$ b	7	16			
	10 B $\rightarrow \bullet$					
3	11 S $\rightarrow$ a $\bullet$ c	6	15			
	12 A $\rightarrow$ a $\bullet$					
4	13 Start $\rightarrow$ S $\bullet$ \$	5	14			
5	14 Start $\rightarrow$ S \$ $\bullet$					
6	15 S $\rightarrow$ a c $\bullet$					
7	16 B $\rightarrow$ b $\bullet$					
8	17 S $\rightarrow$ A B $\bullet$					
9	18 S $\rightarrow$ x A $\bullet$ c	11	20			
10	19 A $\rightarrow$ a $\bullet$					
11	20 S $\rightarrow$ x A c $\bullet$					

Item	Prop To	Initial	Pass 1
1	13	\$	
2	5,8	\$	
3	11	\$	
4	6	\$	
5	12	b	\$
6	18		\$
7	19	c	
8	9,10,17		\$
9	16		\$
10			\$
11	15		\$
12			b \$
13	14		\$
14			\$
15			\$
16			\$
17			\$
18	20		\$
19			c
20			\$

LR(k) Table Construction

1 Start $\rightarrow$ S \$
 2 S $\rightarrow$ lp M rp
 3 | lb M rb
 4 | lp U rb
 5 | lb U rp
 6 M $\rightarrow$ expr
 7 U $\rightarrow$ expr

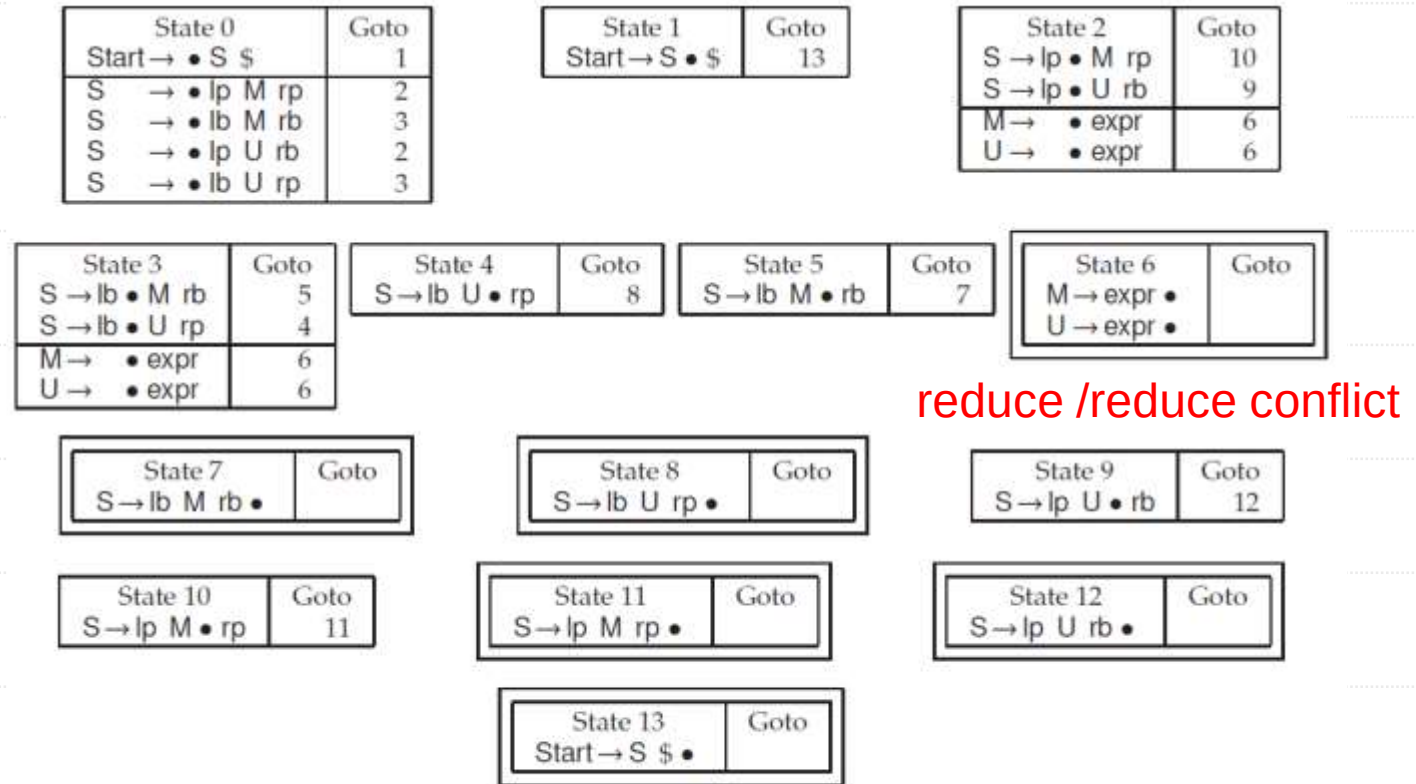


Figure 6.36: LR(0) construction.

LR(k) Table Construction

State	LR(0) Item	Goto State	Prop Edges Placed by Step		Initialize ItemFollow	
			(27)	(29)	First(γ)	(28)
0	1 Start $\rightarrow \bullet$ S \$	1	??		\$	2,3,4,5
	2 S $\rightarrow \bullet$ lp M rp	2	6			
	3 S $\rightarrow \bullet$ lb M rb	3	10			
	4 S $\rightarrow \bullet$ lp U rb	2	7			
	5 S $\rightarrow \bullet$ lb U rp	3	11			
2	6 S $\rightarrow$ lp $\bullet$ M rp	10	??		rp	8
	7 S $\rightarrow$ lp $\bullet$ U rb	9	??		rb	9
	8 M $\rightarrow \bullet$ expr	6	14			
	9 U $\rightarrow \bullet$ expr	6	15			
3	10 S $\rightarrow$ lb $\bullet$ M rb	5	??		rb	12
	11 S $\rightarrow$ lb $\bullet$ U rp	4	??		rp	13
	12 M $\rightarrow \bullet$ expr	6	14			
	13 U $\rightarrow \bullet$ expr	6	15			
6	14 M $\rightarrow$ expr $\bullet$					
	15 U $\rightarrow$ expr $\bullet$					

$ItemFollow(14)$
 $= ItemFollow(15)$
 $= \{ rb, rp \}$

LR(k) Table Construction

1 Start $\rightarrow$ S \$
 2 S $\rightarrow$ lp M rp
 3 | lb M rb
 4 | lp U rb
 5 | lb U rp
 6 M $\rightarrow$ expr
 7 U $\rightarrow$ expr

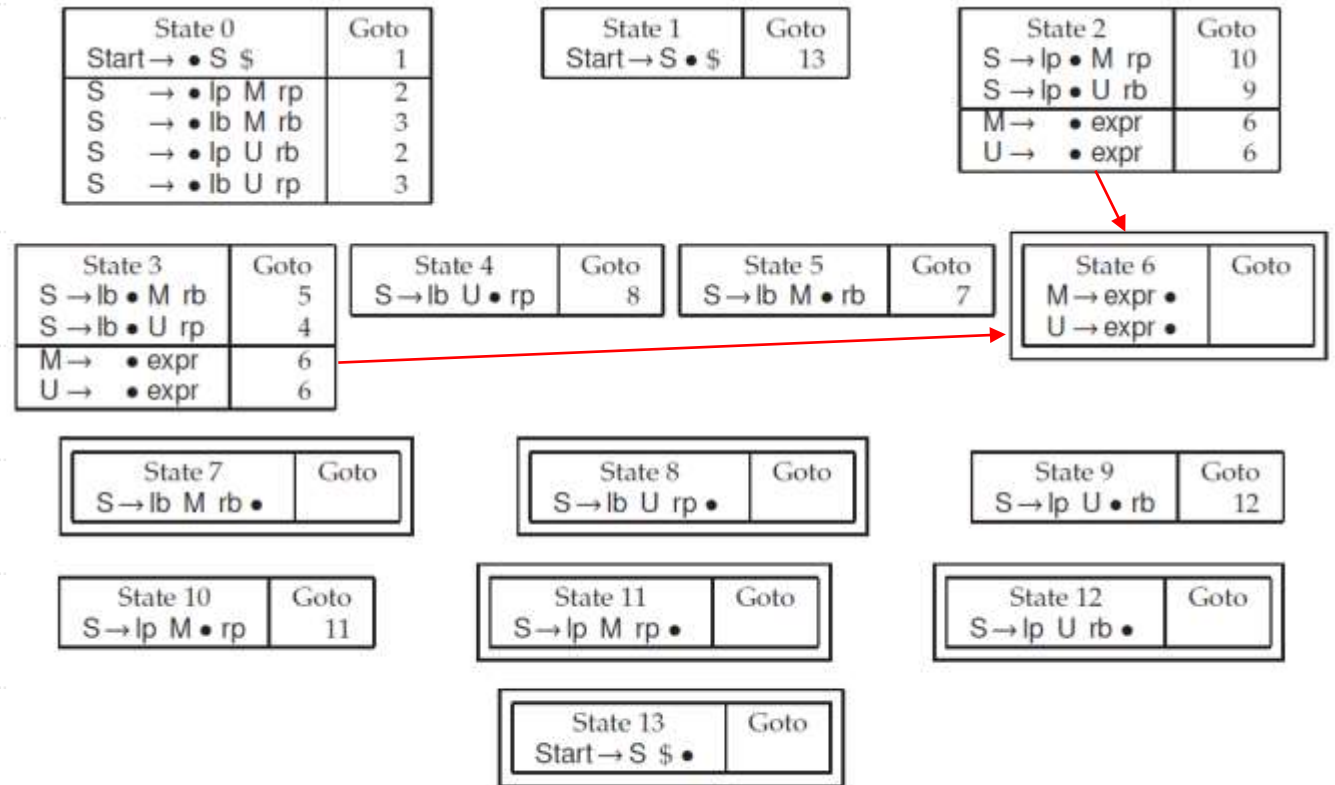


Figure 6.36: LR(0) construction.

LR(k) Table Construction

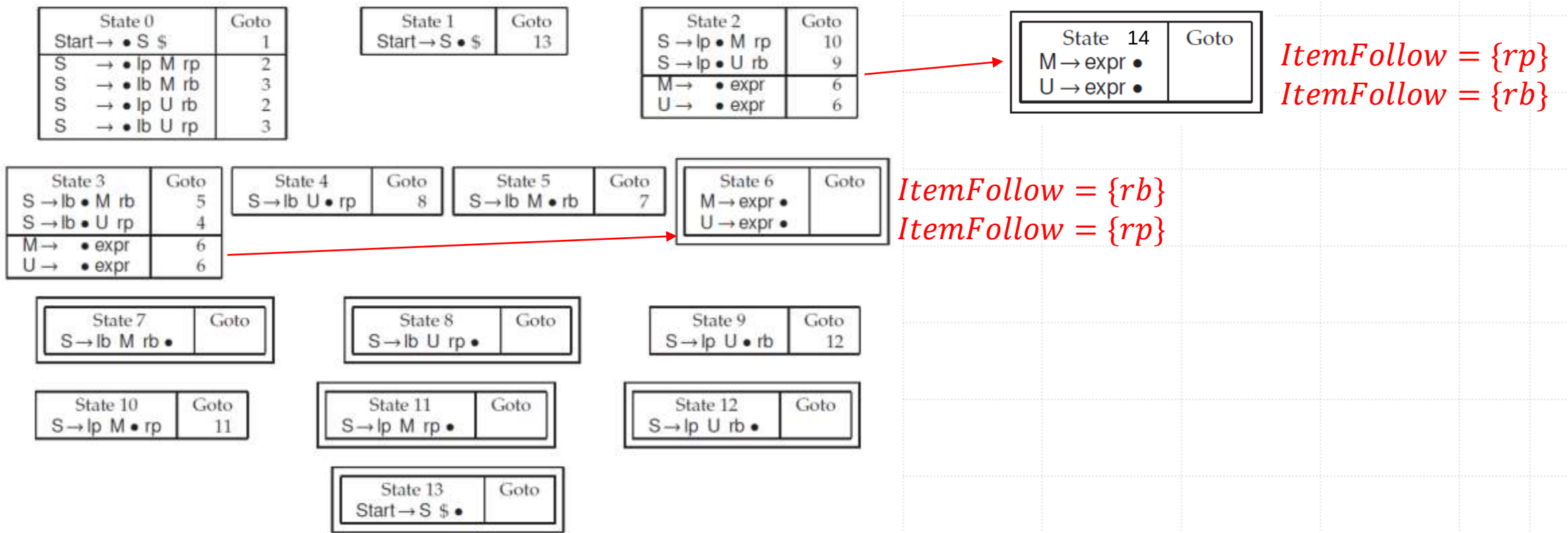


Figure 6.36: LR(0) construction.



LR(k) Table Construction

- For LR(k), we extend an item's notation from $A \rightarrow \alpha \bullet \beta$ to $[A \rightarrow \alpha \bullet \beta, w]$.
- For LR(1), w is a (terminal) symbol that can follow A when this item becomes reducible.
- For LR(k), $k \geq 0$, w is a k -length string that can follow A after reduction.
- If symbols x and y can both follow A when $A \rightarrow \alpha \bullet \beta$ becomes reducible, then the corresponding LR(1) state contains both $[A \rightarrow \alpha \bullet \beta, x]$ and $[A \rightarrow \alpha \bullet \beta, y]$.
- Notice how nicely the notation for LR(k) generalizes LR(0). For LR(0), w must be a 0-length string. The only such string is λ , which provides no information at a possible point of reduction, since λ does not occur as input.

LR(k) Table Construction

State 3	Goto
$[S \rightarrow lb \bullet M \text{ } rb, \$]$	5
$[S \rightarrow lb \bullet U \text{ } rp, \$]$	4
$[M \rightarrow \bullet \text{ } expr, rb]$	14
$[U \rightarrow \bullet \text{ } expr, rp]$	14

$[S \rightarrow lb \bullet M \text{ } rb, \$]$ is not ready for reduction, but indicates that \$ will follow the reduction to S when the item eventually becomes reducible

State 6	Goto
$[M \rightarrow expr \bullet, rp]$	
$[U \rightarrow expr \bullet, rb]$	

The item calls for a reduction by rule $M \rightarrow expr$ when rp is the next input token.

```

function COMPUTELR0( Grammar ) returns ( Set, State )
    States  $\leftarrow \emptyset$ 
    StartItems  $\leftarrow \{ \text{Start} \rightarrow \bullet \text{RHS}(p) \mid p \in \text{PRODUCTIONSFOR}(\text{Start}) \}$  ⑦
    StartState  $\leftarrow \text{ADDSTATE}(\text{States}, \text{StartItems})$ 
    while ( s  $\leftarrow \text{WorkList.EXTRACTELEMENT}()$  )  $\neq \perp$  do
        call COMPUTEGOTO( States, s )
    return ((States, StartState))
end

function ADDSTATE( States, items ) returns State
    if items  $\notin \text{States}$ 
    then
        s  $\leftarrow \text{newState}(\text{items})$ 
        States  $\leftarrow \text{States} \cup \{s\}$ 
        WorkList  $\leftarrow \text{WorkList} \cup \{s\}$ 
        Table[s][ $\star$ ]  $\leftarrow \text{error}$ 
    else s  $\leftarrow \text{FindState}(\text{items})$ 
    return (s)
end

function ADVANCEDOT( state, X ) returns Set
    return (  $\{ A \rightarrow \alpha X \bullet \beta \mid A \rightarrow \alpha \bullet X \beta \in \text{state} \}$  )
end

```

Marker ⑦: We initialize *StartItems* by including LR(1) items that have \$ as the follow symbol:

$$\text{StartItems} \leftarrow \{ [\text{Start} \rightarrow \bullet \text{RHS}(p), \$] \mid p \in \text{PRODUCTIONSFOR}(\text{Start}) \}$$

Marker ⑬: We augment the LR(0) item so that ADVANCEDOT returns the appropriate LR(1) items:

$$\text{return } (\{ [A \rightarrow \alpha X \bullet \beta, a] \mid [A \rightarrow \alpha \bullet X \beta, a] \in \text{state} \})$$

Marker ⑮: This entire loop is replaced by the following:

```

⑨      foreach  $[ A \rightarrow \alpha \bullet B \gamma, a ] \in \text{ans}$  do
        foreach  $p \in \text{PRODUCTIONSFOR}(B)$  do
            foreach  $b \in \text{First}(\gamma a)$  do
                 $\text{ans} \leftarrow \text{ans} \cup \{ [ B \rightarrow \bullet \text{RHS}(p), b ] \}$ 

```

③①

⑪

⑫

⑬

function CLOSURE(*state*) **returns** Set

ans $\leftarrow$ *state*

repeat

prev $\leftarrow$ *ans*

foreach $A \rightarrow \alpha \bullet B\gamma \in \text{ans}$ **do**

foreach $p \in \text{PRODUCTIONS_FOR}(B)$ **do**

ans $\leftarrow \text{ans} \cup \{B \rightarrow \bullet \text{RHS}(p)\}$

until *ans* = *prev*

return (*ans*)

end

procedure COMPUTEGOTO(*States*, *s*)

closed $\leftarrow$ CLOSURE(*s*)

foreach $X \in (N \cup \Sigma)$ **do**

RelevantItems $\leftarrow$ ADVANCEDOT(*closed*, *X*)

if *RelevantItems* $\neq \emptyset$

then

Table[*s*][*X*] $\leftarrow$ shift ADDSTATE(*States*, *RelevantItems*)

end

(14)

(15)

(16)

(17)

(18)

(19)

(20)

Marker ⑦: We initialize *StartItems* by including LR(1) items that have \$ as the follow symbol:

$\text{StartItems} \leftarrow \{[\text{Start} \rightarrow \bullet \text{RHS}(p), \$] \mid p \in \text{PRODUCTIONS_FOR}(\text{Start})\}$

Marker ⑬: We augment the LR(0) item so that ADVANCEDOT returns the appropriate LR(1) items:

$\text{return} (\{[A \rightarrow \alpha X \bullet \beta, a] \mid [A \rightarrow \alpha \bullet X\beta, a] \in \text{state} \})$

Marker ⑮: This entire loop is replaced by the following:

foreach $[A \rightarrow \alpha \bullet B\gamma, a] \in \text{ans}$ **do**

foreach $p \in \text{PRODUCTIONS_FOR}(B)$ **do**

foreach $b \in \text{First}(\gamma a)$ **do**

ans $\leftarrow \text{ans} \cup \{[B \rightarrow \bullet \text{RHS}(p), b]\}$

(31)

LR(k) Table Construction

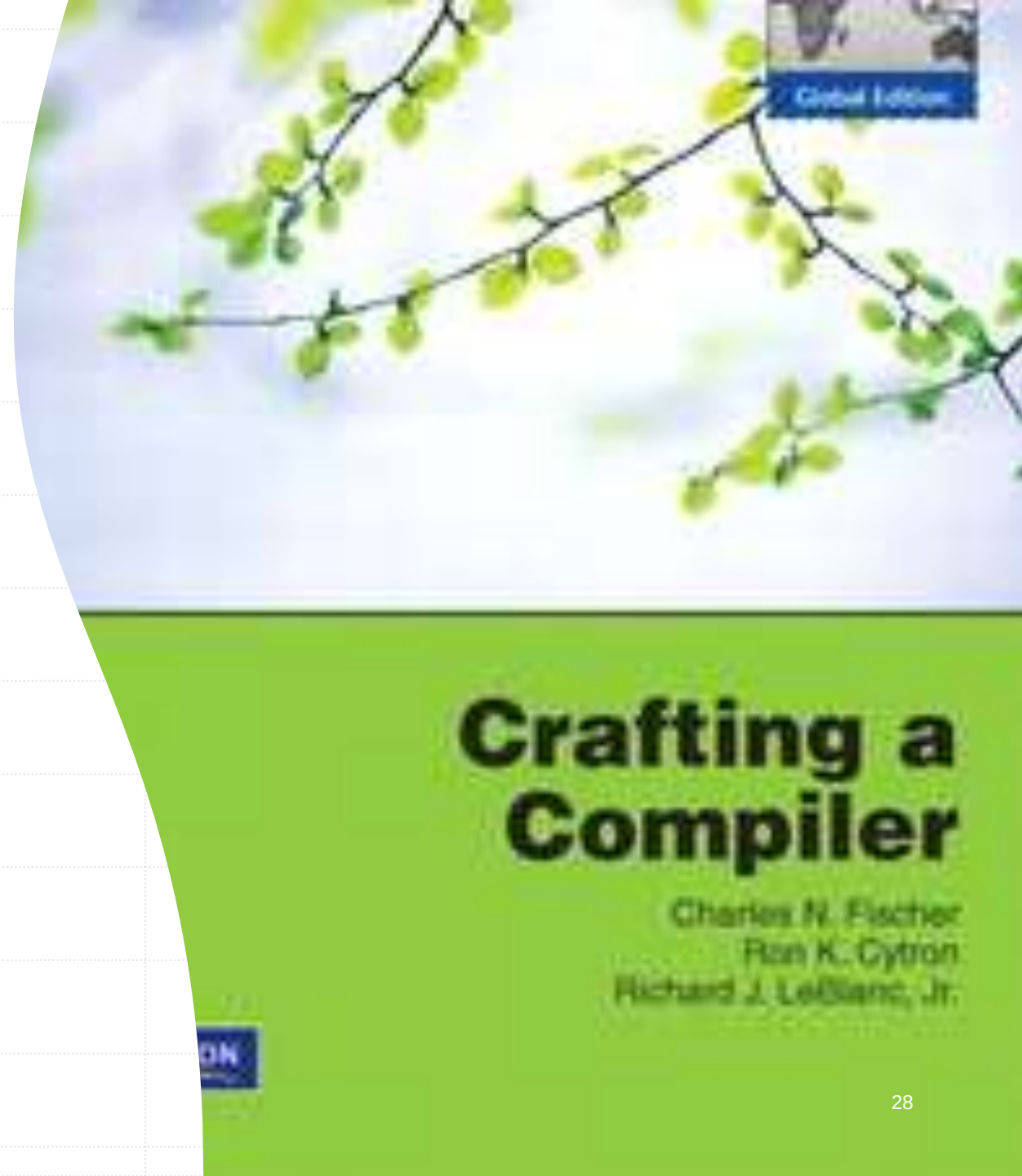
```
procedure COMPLETETABLE(Table, grammar)  
  call COMPUTELOOKAHEAD(-)  
  foreach state  $\in$  Table do  
    foreach rule  $\in$  Productions(grammar) do  
      call TRYRULEINSTATE(state, rule)  
    call ASSERTENTRY(StartState, GoalSymbol, accept)  
  end  
  procedure ASSERTENTRY(state, symbol, action)  
    if Table[state][symbol] = error  
    then Table[state][symbol]  $\leftarrow$  action  
    else  
      call REPORTCONFLICT(Table[state][symbol], action)  
    end  
  end
```

```
procedure TRYRULEINSTATE(s, r)  
  if  $\text{LHS}(r) \rightarrow \text{RHS}(r) \bullet \in s$   
  then  
    foreach  $\mathcal{X} \in \text{Follow}(\text{LHS}(r))$  do  
      call ASSERTENTRY(s,  $\mathcal{X}$ , reduce r)  
    end  
  end
```

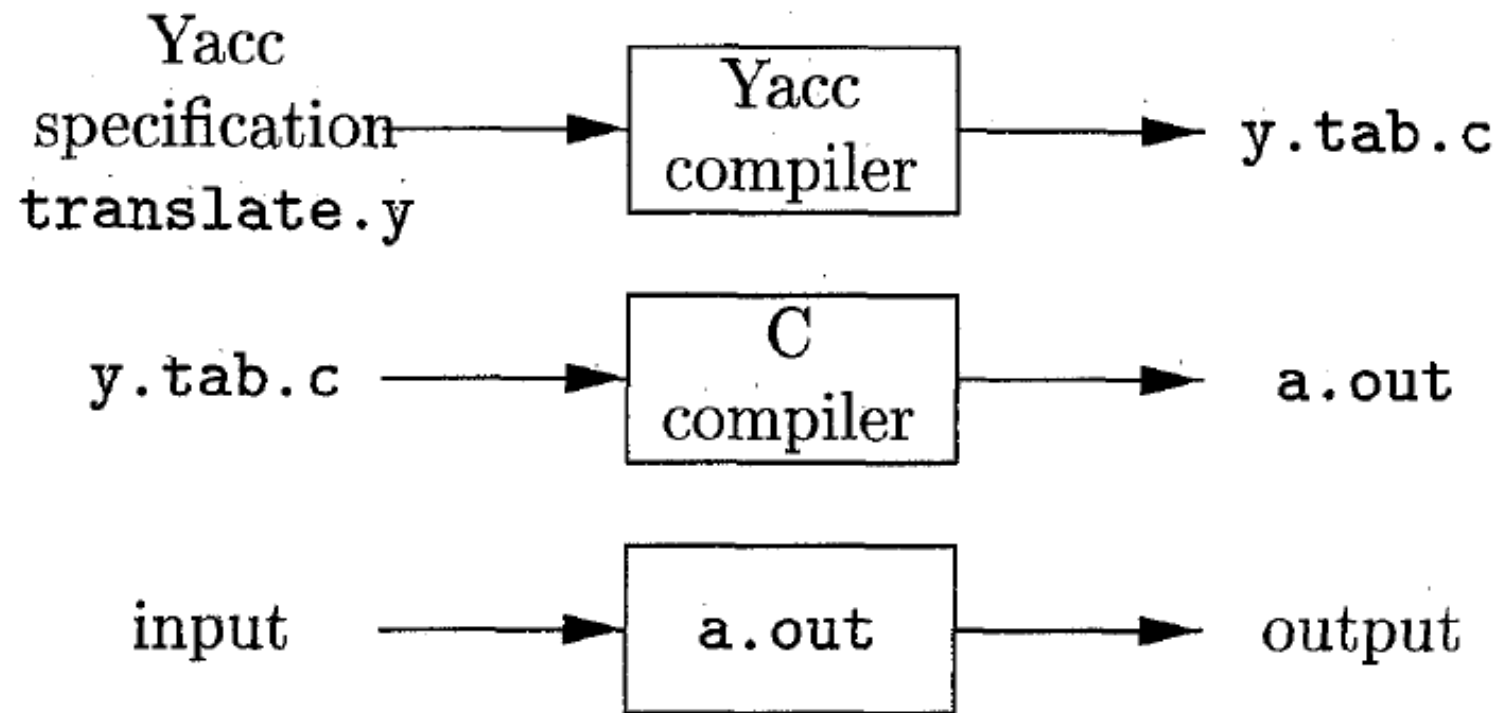


```
procedure TRYRULEINSTATE(s, r)  
  if  $[\text{LHS}(r) \rightarrow \text{RHS}(r) \bullet, w] \in s$   
  then call ASSERTENTRY(s, w, reduce r)  
  end
```


- Exercises 27, 40, 41, 45



Yacc





Disambiguating Rules for Yacc

(*required only when there exists a conflict)

1. In a shift/reduce conflict the default is to shift.
2. In a reduce/reduce conflict the default is to reduce by the earlier grammar rule in the input sequence.
3. Precedence and associativity (left, right, nonassoc) are recorded for each token that have them.

- 
4. Precedence and associativity of a production rule is that (if any) of its final (rightmost) token unless a "%prec " overrides. Then it is the token given following %prec.
 5. In a shift/reduce conflict where both the grammar rule and the input (lookahead) have precedence, resolve in favor of the rule of higher precedence. In a tie, use associativity. That is, left assoc. => reduce; right assoc. => shift; nonassoc => error.
 6. Otherwise use 1 and 2.

(Please See Page 238 of the Textbook)

Yacc

declarations

%%

translation rules

%%

supporting C routines

$\langle \text{head} \rangle$: $\langle \text{body} \rangle_1$ { $\langle \text{semantic action} \rangle_1$ }
| $\langle \text{body} \rangle_2$ { $\langle \text{semantic action} \rangle_2$ }
...
| $\langle \text{body} \rangle_n$ { $\langle \text{semantic action} \rangle_n$ }
;

```
%{
#include <ctype.h>
%}

%token DIGIT → declared in "y.tab.h"

%%
line   : expr '\n'          { printf("%d\n", $1); }
      ;
expr   : expr '+' term      { $$ = $1 + $3; }
      | term
      ;
term   : term '*' factor    { $$ = $1 * $3; }
      | factor
      ;
factor : '(' expr ')'       { $$ = $2; }
      | DIGIT               { $$ = $1 }
      ;

%%
yylex() {
    int c;
    c = getchar();
    if (isdigit(c)) {
        yylval = c - '0';
        return DIGIT;
    }
    return c;
}
```



Yacc

- The lexical analyzer `yylex()` produces tokens consisting of a token name and its associated attribute value. If a token name such as `DIGIT` is returned, the token name must be declared in the first section of the Yacc specification. The attribute value associated with a token is communicated to the parser through a Yacc-defined variable `yylval`.
- Whenever the lexer returns a token to the parser, if the token has an associated value, the lexer must store the value in `yylval` before returning. In this first example, we explicitly declare `yylval`. In more complex parsers, yacc defines `yylval` as a union and puts the definition in `y.tab.h`.

Yacc

```
%{  
#include "y.tab.h"  
extern int yylval;  
%}  
  
%%  
[0-9]+ { yylval = atoi(yytext); return NUMBER; }  
[ \t] ; /* ignore whitespace */  
\n return 0; /* logical EOF */  
.  
return yytext[0];  
%%
```

declared in "y.tab.h"

declared by yacc

```
%token NAME NUMBER  
%%  
statement: NAME '=' expression  
          | expression { printf("= %d\n", $1); }  
          ;  
  
expression: expression '+' NUMBER { $$ = $1 + $3; }  
           | expression '-' NUMBER { $$ = $1 - $3; }  
           | NUMBER { $$ = $1; }  
           ;
```

The Lexer

A Yacc Parser



Yacc

- On a UNIX system, yacc takes your grammar and creates y.tab.c, the C language parser, and y.tab.h, the include file with the token number definitions. Lex creates lex.yy.c, the C language lexer. You need only compile them together with the yacc and lex libraries. The libraries contain usable default versions of all of the supporting routines, including a **main() that calls the parser yyparse() and exits.**

```
% yacc -d ch3-01.y    # makes y.tab.c and "y.tab.h
```

```
% lex ch3-01.l        # makes lex.yy.c
```

```
% cc -o ch3-01 y.tab.c lex.yy.c -ly -ll    # compile and link C files
```

Yacc

```
%token NAME NUMBER
%left '-' '+'
%left '*' '/'
%nonassoc UMINUS

%%

statement: NAME '=' expression
          | expression          { printf("= %d\n", $1); }
          ;

expression: expression '+' expression { $$ = $1 + $3; }
          | expression '-' expression { $$ = $1 - $3; }
          | expression '*' expression { $$ = $1 * $3; }
          | expression '/' expression
            { if($3 == 0)
              yyerror("divide by zero");
              else
                $$ = $1 / $3;
            }

          | '-' expression %prec UMINUS { $$ = -$2; }
          | '(' expression ')'          { $$ = $2; }
          | NUMBER                      { $$ = $1; }
          ;

%%
```

```

%{
double vbltable[26];
%{

%union {
    double dval;
    int vblno;
}

%token <vblno> NAME
%token <dval> NUMBER
%left '-' '+'
%left '*' '/'
%nonassoc UMINUS

%type <dval> expression
%%
statement_list: statement '\n'
               | statement_list statement '\n'
               ;

statement: NAME '=' expression { vbltable[$1] = $3; }
          | expression { printf("= %g\n", $1); }
          ;

expression: expression '+' expression { $$ = $1 + $3; }
           | expression '-' expression { $$ = $1 - $3; }
           | expression '*' expression { $$ = $1 * $3; }
           | expression '/' expression
             { if($3 == 0.0)
                 yyerror("divide by zero")
               else
                 $$ = $1 / $3;
             }
           | '-' expression %prec UMINUS { $$ = -$2; }

```

```

           | '(' expression ')' { $$ = $2; }
           | NUMBER
           | NAME { $$ = vbltable[$1]; }
;
%%

```

Example 3-3. Lexer for calculator with variables and real values ch3-03.1

```

%{
#include "y.tab.h"
#include <math.h>
extern double vbltable[26];
%}

%%
([0-9]+|([0-9]*\.[0-9]+)([eE][+-]?[0-9]+)?) {
    yylval.dval = atof(yytext); return NUMBER;
}

[ \t] ; /* ignore whitespace */

[a-z] { yylval.vblno = yytext[0] - 'a'; return NAME; }

"$" { return 0; /* end of input */ }

\n |
. return yytext[0];
%%

```



Yacc

- The generated header file y.tab.h includes a copy of the definition so that you can use it in the lexer. Here is the y.tab.h generated from this grammar:

```
#define NAME 257
#define NUMBER 258
#define UMINUS 259
typedef union {
    double dval;
    int vblno;
} YYSTYPE;
extern YYSTYPE yylval;
```

Symbol table

Example 3-4. Header for parser with symbol table ch3hdr.h

```
#define NSYMS 20 /* maximum number of symbols */
```

```
struct symtab {  
    char *name;  
    double value;  
} symtab[NSYMS];
```

```
struct symtab *symlook();
```

```
/* look up a symbol table entry, add if not present */  
struct symtab *  
symlook(s)  
char *s;  
{  
    char *p;  
    struct symtab *sp;  
    for(sp = symtab; sp < &symtab[NSYMS]; sp++) {  
        /* is it already here? */  
        if(sp->name && !strcmp(sp->name, s))  
            return sp;  
  
        /* is it free */  
        if(!sp->name) {  
            sp->name = strdup(s);  
            return sp;  
        }  
        /* otherwise continue to next */  
    }  
    yyerror("Too many symbols");  
    exit(1); /* cannot continue */  
} /* symlook */
```



```

%{
#include "ch3hdr.h"
#include <string.h>
%}

%union {
    double dval;
    struct syntab *symp;
}

%token <symp> NAME
%token <dval> NUMBER
%left '-' '+'
%left '*' '/'
%nonassoc UMINUS

%type <dval> expression
%%

statement_list: statement '\n'
               | statement_list statement '\n'
               %%

```

```

;

statement: NAME '=' expression { $1->value = $3; }
         | expression          { printf("= %g\n", $1); }
         ;

expression: expression '+' expression { $$ = $1 + $3; }
          | expression '-' expression { $$ = $1 - $3; }
          | expression '*' expression { $$ = $1 * $3; }
          | expression '/' expression
            {
              if($3 == 0.0)
                yyerror("divide by zero")
              else
                $$ = $1 / $3;
            }
          | '-' expression %prec UMINUS { $$ = -$2; }
          | '(' expression ')'          { $$ = $2; }
          | NUMBER
          | NAME                        { $$ = $1->value; }
          ;

```



Symbol table (Lex)

```
%{
#include "y.tab.h"
#include "ch3hdr.h"
#include <math.h>
%}

%%
([0-9]+|([0-9]*\.[0-9]+)([eE][+-]?[0-9]+)?) {
    yylval.dval = atof(yytext);
    return NUMBER;
}
[ \t] ;          /* ignore whitespace */

[A-Za-z][A-Za-z0-9]* { /* return symbol pointer */
    yylval.symp = symlook(yytext);
    return NAME;
}

"$" { return 0; }
\n |
.   return yytext[0];
%%
```